

LAPORAN PRAKTIKUM STRUKTUR DATA

PERTEMUAN 8

Algoritma Sorting (Shell, Quick, dan Merge)



OLEH:

LEXI MULIA YUNASPI

2511531006

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : Dr, WAHYUDI M.T

ASISTEN PRAKTIKUM : M.GHALID

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan Praktikum Struktur Data ini dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan selama perkuliahan. Dalam laporan ini dibahas mengenai penerapan beberapa algoritma sorting, yaitu Bubble Sort, Shell Sort, Quick Sort, dan Merge Sort menggunakan bahasa pemrograman Java. Selain itu, laporan ini juga memuat langkah-langkah pembuatan program, penjelasan kode program, serta hasil pengujian yang telah dilakukan selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi penyusunan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa yang akan datang. Penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data serta semua pihak yang telah memberikan arahan dan bantuan selama pelaksanaan praktikum dan penyusunan laporan ini.

Semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai penerapan algoritma sorting dalam pemrograman Java.

Padang, Juni 2026

Penulis

DAFTAR ISI

PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Langkah Langkah Praktikum	2
2.2 Langkah Langkah Tugas Praktikum	39
BAB III PENUTUP	40
3.1 Kesimpulan	41
3.2 Saran	41
DAFTAR PUSTAKA	42

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi saat ini menuntut kemampuan dalam mengelola dan mengolah data secara cepat dan efisien. Salah satu konsep penting dalam Struktur Data adalah proses pengurutan data (sorting). Pengurutan data bertujuan untuk menyusun data berdasarkan aturan tertentu sehingga data lebih mudah dicari, dianalisis, dan digunakan dalam berbagai kebutuhan pemrograman.

Dalam praktikum ini digunakan beberapa algoritma sorting, yaitu Bubble Sort, Shell Sort, Quick Sort, dan Merge Sort yang diimplementasikan menggunakan bahasa pemrograman Java. Setiap algoritma memiliki cara kerja, kelebihan, dan tingkat efisiensi yang berbeda dalam mengurutkan data. Melalui praktikum ini mahasiswa dapat memahami proses pengurutan data serta membandingkan karakteristik dari masing-masing algoritma sorting.

Selain mempelajari algoritma sorting berbasis console, praktikum juga menerapkan konsep Graphical User Interface (GUI) pada program Bubble Sort sehingga proses pengurutan dapat diamati secara visual dan lebih mudah dipahami. Dengan demikian mahasiswa dapat memperoleh pemahaman yang lebih baik mengenai penerapan algoritma sorting dalam pengembangan program.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut:

1. Memahami konsep dasar algoritma pengurutan data (sorting).
2. Mengimplementasikan algoritma Bubble Sort, Shell Sort, Quick Sort, dan Merge Sort menggunakan bahasa Java.

3. Memahami cara kerja masing-masing algoritma sorting dalam mengurutkan data.
4. Menerapkan konsep pemrograman berorientasi objek pada program Java.
5. Membuat program GUI sederhana untuk menampilkan proses Bubble Sort secara visual.
6. Menganalisis hasil pengurutan data yang dihasilkan oleh setiap algoritma sorting.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Menambah pemahaman mahasiswa mengenai konsep Struktur Data, khususnya algoritma sorting.
2. Melatih kemampuan mahasiswa dalam membuat program menggunakan bahasa Java.
3. Membantu mahasiswa memahami perbedaan cara kerja dan efisiensi berbagai algoritma sorting.
4. Meningkatkan kemampuan dalam menerapkan konsep Object Oriented Programming (OOP).
5. Memberikan pengalaman dalam membuat aplikasi berbasis GUI menggunakan Java Swing.
6. Menjadi dasar untuk mempelajari algoritma dan struktur data yang lebih kompleks pada materi berikutnya.

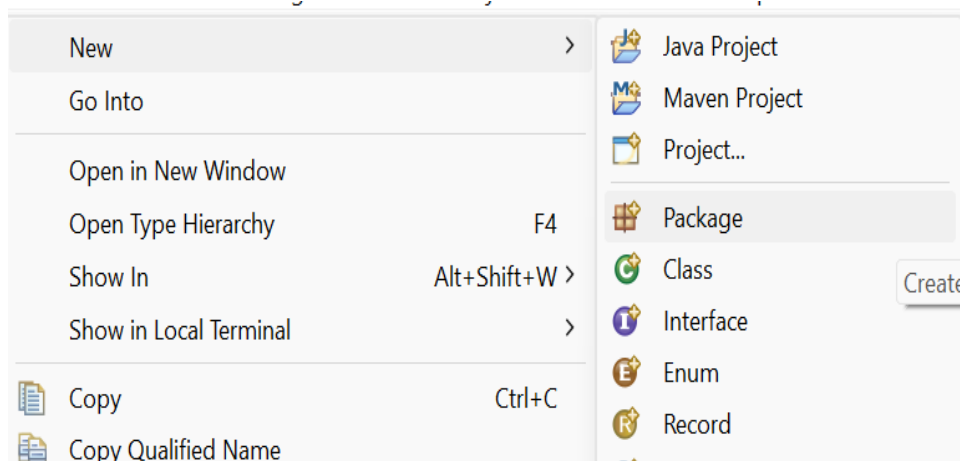
BAB II

PEMBAHASAN

2.1 Langkah Langkah Praktikum

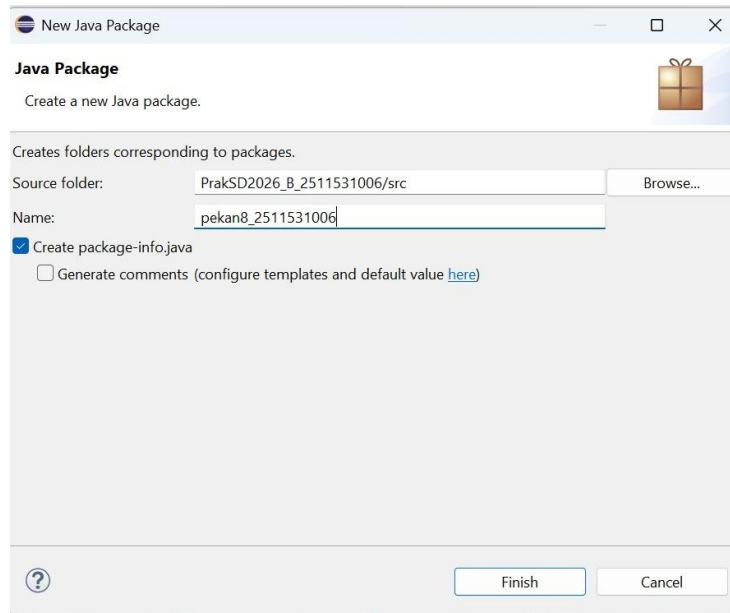
1. Program 1

a. Membuka Eclipse dan Membuat Project



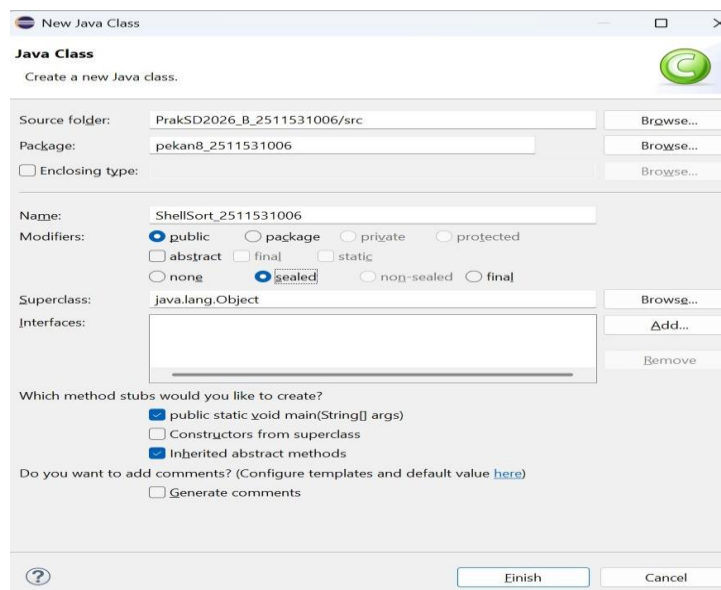
Pada tahap ini dilakukan pembukaan aplikasi Eclipse untuk memulai praktikum. Setelah Eclipse berhasil dijalankan, dibuat project Java yang akan digunakan untuk menulis program. Project berfungsi sebagai tempat penyimpanan seluruh file yang berkaitan dengan praktikum. Dengan adanya project, pengelolaan file menjadi lebih teratur. Setelah project selesai dibuat, proses dapat dilanjutkan ke tahap berikutnya.

b. Membuat Package Java



Pada tahap ini dibuat package dengan nama pekan8_2511531006. Package digunakan untuk mengelompokkan file program agar lebih rapi dan mudah dikelola. Pembuatan package dilakukan dengan memilih menu New → Package pada Eclipse. Setelah nama package diisi sesuai ketentuan, klik tombol Finish. Package yang telah dibuat akan digunakan sebagai tempat penyimpanan class program.

c. Membuat Class Java



Pada tahap ini dibuat class dengan nama ShellSort_2511531006 pada package yang telah dibuat sebelumnya. Class digunakan sebagai tempat menuliskan seluruh kode program Java. Pembuatan class dilakukan melalui menu New → Class. Saat membuat class, dipilih method public static void main(String[] args) agar program dapat dijalankan. Setelah selesai, class siap digunakan untuk menuliskan program Shell Sort.

d. Menuliskan Package Program

```
1 package pekan8_2511531006;  
2
```

Pada bagian ini dituliskan package yang digunakan pada program. Package pekan8_2511531006 merupakan package yang telah dibuat sebelumnya pada Eclipse. Fungsi package adalah untuk mengelompokkan file program agar lebih terstruktur. Dengan adanya package, pengelolaan file menjadi lebih mudah dilakukan. Selain itu package juga membantu menghindari konflik nama class dalam project.

e. Mendeklarasikan Class

```
2  
3 public class ShellSort_2511531006 {  
4
```

Kode di atas digunakan untuk membuat class utama bernama ShellSort_2511531006. Class ini berfungsi sebagai tempat penyimpanan seluruh kode program Shell Sort. Semua method dan variabel yang digunakan dalam program ditulis di dalam class ini. Class merupakan komponen utama dalam pemrograman berorientasi objek. Oleh karena itu setiap program Java harus memiliki class sebagai wadah program.

f. Membuat Method Shell Sort

```
4  
5 public static void shellSort_1006(int[] A_1006) {  
6     int n_1006 = A_1006.length;  
7     int gap_1006 = n_1006 / 2;
```

Method shellSort_1006() digunakan untuk melakukan proses pengurutan data menggunakan algoritma Shell Sort. Variabel n_1006 digunakan untuk menyimpan jumlah elemen yang terdapat pada array. Sedangkan variabel gap_1006 digunakan sebagai jarak antar elemen yang akan dibandingkan. Nilai gap awal diperoleh dari panjang array dibagi dua. Method ini menjadi bagian utama dalam proses sorting.

g. Membuat Perulangan While

```
8 while (gap_1006 > 0) {
```

Perulangan while digunakan untuk menjalankan proses sorting selama nilai gap masih lebih besar dari nol. Pada setiap perulangan, program akan membandingkan elemen-elemen yang memiliki jarak sesuai nilai gap. Setelah satu tahap selesai, nilai gap akan diperkecil. Proses ini dilakukan secara berulang sampai gap bernilai nol. Ketika gap bernilai nol, proses sorting dinyatakan selesai.

h. Membuat Perulangan For

```
9 for (int i_1006 = gap_1006; i_1006 < n_1006; i_1006++) {
```

Perulangan for digunakan untuk menelusuri seluruh elemen array yang akan dibandingkan. Perulangan dimulai dari indeks sebesar nilai gap hingga indeks terakhir array. Setiap elemen akan diperiksa untuk menentukan

apakah perlu dipindahkan atau tidak. Proses ini dilakukan pada setiap tahap perubahan gap. Dengan cara ini seluruh data dapat diperiksa secara bertahap.

i. Menyimpan Data Sementara

```
10         int temp_1006 = A_1006[i_1006];  
11         int j_1006 = i_1006;
```

Variabel temp_1006 digunakan untuk menyimpan nilai data yang sedang diproses. Penyimpanan sementara diperlukan agar data tidak hilang saat terjadi perpindahan elemen. Variabel j_1006 digunakan untuk menyimpan posisi indeks yang sedang dibandingkan. Kedua variabel ini membantu proses penggeseran data dalam algoritma Shell Sort. Dengan demikian proses sorting dapat berjalan dengan benar.

i. Membandingkan dan Menggeser Data

```
12         while (j_1006 >= gap_1006 && A_1006[j_1006 - gap_1006] > temp_1006) {  
13             A_1006[j_1006] = A_1006[j_1006 - gap_1006];
```

Bagian ini digunakan untuk membandingkan data yang sedang diproses dengan data sebelumnya berdasarkan nilai gap. Jika data sebelumnya lebih besar dari data saat ini, maka data tersebut akan digeser ke kanan. Penggeseran dilakukan sampai ditemukan posisi yang sesuai. Proses ini mirip dengan cara kerja Insertion Sort. Tujuannya adalah menempatkan data pada posisi yang benar.

j. Menempatkan Data pada Posisi yang Tepat

```
         A_1006[j_1006] = temp_1006;
```

Setelah posisi yang sesuai ditemukan, data yang disimpan pada variabel temp_1006 akan ditempatkan kembali ke dalam array. Penempatan dilakukan pada indeks yang telah ditentukan selama proses perbandingan. Langkah ini memastikan data berada pada posisi

yang benar. Proses ini dilakukan pada setiap elemen yang diperiksa. Hasilnya adalah susunan data yang semakin terurut.

k. Mengubah Nilai Gap

```
gap_1006 = gap_1006 / 2;
```

Kode ini digunakan untuk memperkecil nilai gap setelah satu tahap sorting selesai dilakukan. Nilai gap akan berubah dari 5 menjadi 2 kemudian menjadi 1. Semakin kecil nilai gap maka proses pengurutan akan semakin detail. Ketika gap bernilai satu proses sorting hampir selesai dilakukan. Setelah gap menjadi nol maka perulangan akan berhenti.

l. Membuat Data Array

```
23 int[] data_1006 = {1, 10, 4, 6, 8, 9, 7, 2, 1, 5};  
24
```

Array digunakan sebagai tempat penyimpanan data yang akan diurutkan. Data yang dimasukkan masih dalam kondisi belum terurut. Jumlah data yang digunakan sebanyak sepuluh elemen bertipe integer. Data ini nantinya akan diproses menggunakan algoritma Shell Sort. Hasil pengurutan akan ditampilkan pada layar console.

m. Menampilkan Data Sebelum Sorting

```
24  
25 System.out.print("Sebelum : ");  
26 printArray_1006(data_1006);  
27
```

```
27  
28     shellSort_1006(data_1006);  
29
```

Perintah ini digunakan untuk memanggil method shellSort 1006() yang telah dibuat

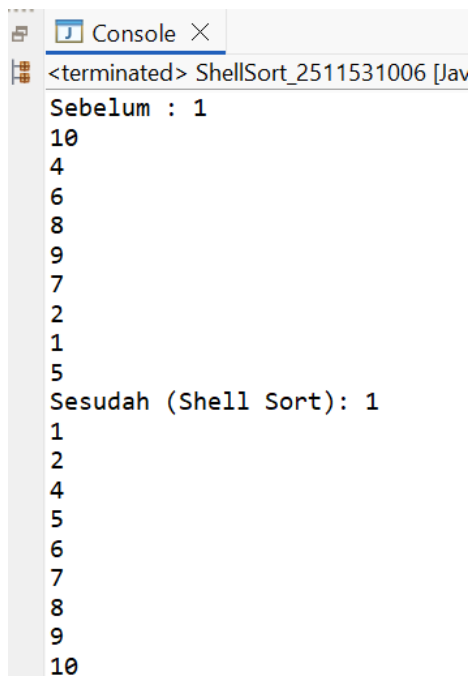
```
29
30     System.out.print("Sesudah (Shell Sort): ");
31     printArray_1006(data_1006);
32 }
```

Kode ini digunakan untuk menampilkan hasil pengurutan setelah proses sorting selesai

```
33  
34 public static void printArray_1006(int[] arr_1006) {  
35
```

Method `printArray_1006()` digunakan untuk menampilkan seluruh isi array ke layar. Method ini menerima parameter berupa array yang akan dicetak. Setiap elemen array dibaca menggunakan perulangan `for-each`. Nilai yang diperoleh kemudian ditampilkan menggunakan perintah `System.out.print()`. Method ini digunakan sebelum dan sesudah proses sorting.

q. Hasil output program

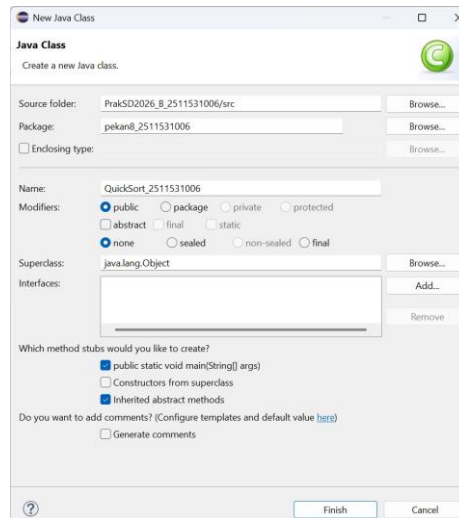


```
<terminated> ShellSort_2511531006 [Jav
Sebelum : 1
10
4
6
8
9
7
2
1
5
Sesudah (Shell Sort): 1
1
2
4
5
6
7
8
9
10
```

Pada tahap ini program dijalankan untuk melihat hasil proses pengurutan data. Sebelum dilakukan sorting, data ditampilkan dalam kondisi awal sesuai isi array yang dimasukkan. Setelah method `shellSort_1006()` dijalankan, data berhasil diurutkan dari nilai terkecil ke terbesar. Hasil output menunjukkan bahwa algoritma Shell Sort bekerja dengan baik sesuai tujuan program. Dengan demikian proses pengurutan data dapat dilakukan dengan benar tanpa mengalami error.

2. Program 2

a. Membuat Class Java



Pada tahap ini dibuat class dengan nama QuickSort_2511531006 pada package yang telah dibuat sebelumnya. Class digunakan sebagai tempat untuk menuliskan seluruh kode program yang akan dijalankan. Pembuatan class dilakukan melalui menu New → Class pada Eclipse. Pada saat pembuatan class dipilih method public static void main(String[] args) agar program dapat dieksekusi. Setelah proses selesai, class siap digunakan untuk menuliskan program Quick Sort.

b. Menuliskan Package Program

```
1 package pekan8_2511531006;  
2
```

Pada bagian ini dituliskan package yang digunakan pada program yaitu pekan8_2511531006. Package berfungsi untuk mengelompokkan file program agar lebih terstruktur dan mudah dikelola. Penulisan package harus sesuai dengan package yang telah dibuat sebelumnya. Dengan adanya package, file program dapat tersusun dengan lebih rapi. Selain itu package juga membantu menghindari konflik nama class dalam project.

c. Mendeklarasikan Class

```
3 public class QuickSort_2511531006 {  
4
```

Pada bagian ini dibuat class utama dengan nama QuickSort_2511531006. Class digunakan sebagai wadah untuk menyimpan seluruh kode program Quick Sort yang akan dibuat. Semua method dan variabel yang digunakan dalam program dituliskan di dalam class ini. Class merupakan komponen utama dalam pemrograman berorientasi objek pada Java. Oleh karena itu setiap program Java harus memiliki class agar dapat dijalankan.

d. Membuat Method Swap

```
5 static void swap_1006(int[] arr_1006, int i_1006, int j_1006)  
6 {  
7     int temp_1006 = arr_1006[i_1006];  
8     arr_1006[i_1006] = arr_1006[j_1006];  
9     arr_1006[j_1006] = temp_1006;  
10 }
```

Pada bagian ini dibuat method swap_1006() yang digunakan untuk menukar posisi dua elemen dalam array. Variabel temp_1006 digunakan sebagai tempat penyimpanan sementara saat proses pertukaran berlangsung. Setelah nilai pertama disimpan, nilai kedua dipindahkan ke posisi pertama. Kemudian nilai yang disimpan pada variabel sementara ditempatkan pada posisi kedua. Method ini sering digunakan selama proses Quick Sort berlangsung.

e. Membuat Method Median Of Three

```
11  
12 // Metode untuk mengatur pivot menggunakan Median-of-Three  
13 static void medianOfThree_1006(int[] arr_1006, int low_1006, int high_1006)
```

Pada bagian ini dibuat method medianOfThree_1006() yang digunakan untuk menentukan pivot menggunakan metode Median of Three. Metode ini memilih pivot dari tiga elemen yaitu elemen awal, tengah, dan akhir array. Tujuannya adalah memperoleh pivot yang lebih baik sehingga proses pengurutan menjadi lebih efisien. Method ini akan dipanggil sebelum proses partition dilakukan. Dengan demikian peluang mendapatkan pembagian data yang seimbang menjadi lebih.

f. Menentukan Nilai Tengah Array

```
15 int mid_1006 = low_1006 + (high_1006 - low_1006) / 2;  
16
```

Pada bagian ini dibuat variabel mid_1006 yang digunakan untuk menentukan posisi tengah array. Nilai tengah diperoleh dari indeks awal dan indeks akhir array. Posisi ini digunakan dalam proses pemilihan pivot menggunakan metode Median of Three. Data pada posisi awal, tengah, dan akhir akan dibandingkan. Hasil perbandingan digunakan untuk menentukan nilai median sebagai pivot.

g. Membandingkan dan Menukar Data

```
18 if (arr_1006[low_1006] > arr_1006[mid_1006]) {  
19     swap_1006(arr_1006, low_1006, mid_1006);  
20 }
```

Pada bagian ini dilakukan perbandingan antara data pertama dan data tengah. Jika data pertama lebih besar dari data tengah maka kedua data akan ditukar posisinya. Pertukaran dilakukan menggunakan method `swap_1006()` yang telah dibuat sebelumnya. Langkah ini merupakan bagian dari proses pemilihan pivot. Tujuannya agar data lebih teratur sebelum proses partition dilakukan.

i. Membuat Method Partition

```
29 static int partition_1006(int[] arr_1006, int low_1006, int high_1006)  
30 {
```

Pada bagian ini dibuat method `partition_1006()` yang digunakan untuk membagi array menjadi dua bagian berdasarkan pivot. Data yang lebih kecil dari pivot akan ditempatkan di sebelah kiri. Sedangkan data yang lebih besar akan ditempatkan di sebelah kanan. Pembagian ini dilakukan sebelum proses rekursif dijalankan. Method `partition` merupakan inti dari algoritma Quick Sort.

h. Menentukan Pivot

```
33  
34 int pivot_1006 = arr_1006[high_1006];
```

Pada bagian ini dibuat variabel `pivot_1006` yang digunakan untuk menyimpan nilai pivot. Pivot merupakan nilai acuan yang digunakan untuk membandingkan seluruh data dalam array. Nilai pivot diambil dari elemen terakhir setelah proses Median of Three dilakukan. Setiap elemen akan dibandingkan dengan pivot tersebut. Hasil perbandingan menentukan posisi akhir data.

i. Membuat Perulangan For

```
36  
37 for (int j_1006 = low_1006; j_1006 <= high_1006 - 1; j_1006++) {
```

Pada bagian ini digunakan perulangan for untuk memeriksa seluruh elemen array. Setiap elemen dibandingkan dengan nilai pivot yang telah ditentukan. Jika elemen lebih kecil dari pivot maka elemen akan dipindahkan ke bagian kiri array. Proses ini dilakukan sampai seluruh data selesai diperiksa. Dengan cara ini array dapat dibagi menjadi dua bagian sesuai aturan Quick Sort.

j. Membuat Method Quick Sort

```
49  
50 static void quickSort_1006(int[] arr_1006, int low_1006, int high_1006) {  
51
```

Pada bagian ini dibuat method quickSort_1006() yang merupakan method utama dalam algoritma Quick Sort. Method ini akan memanggil fungsi partition untuk menentukan posisi pivot. Setelah pivot ditemukan, array dibagi menjadi dua bagian. Kedua bagian tersebut kemudian diurutkan kembali secara rekursif. Proses berlangsung hingga seluruh data tersusun dengan benar.

k. Membuat Method Print Array

Pada bagian ini dibuat method printArray_1006() yang digunakan untuk menampilkan isi array ke layar. Setiap elemen array dibaca menggunakan perulangan for. Data yang dibaca kemudian dicetak menggunakan perintah System.out.print(). Method ini digunakan sebelum dan sesudah proses sorting. Dengan demikian hasil pengurutan dapat diamati dengan jelas.

```
58  
59 static void printArray_1006(int[] arr_1006) {  
60
```

l. Membuat Data Array

```
69 int[] arr_1006 = {10, 7, 8, 9, 1, 5};
```

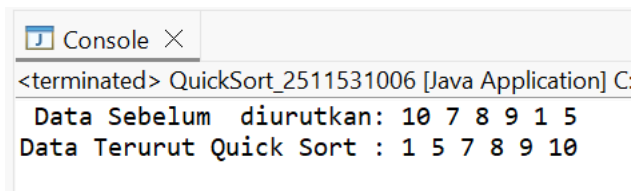
Pada bagian ini dibuat sebuah array yang digunakan sebagai data masukan program. Data yang dimasukkan masih dalam kondisi belum terurut. Seluruh elemen array akan diproses menggunakan algoritma Quick Sort. Data tersebut terdiri dari enam elemen bertipe integer. Hasil pengurutan akan ditampilkan pada layar console.

m. Menjalankan Quick Sort

```
73  
74 quickSort_1006(arr_1006, 0, N_1006 - 1);
```

Pada bagian ini method `quickSort_1006()` dipanggil untuk menjalankan proses pengurutan data. Program akan melakukan proses partition dan rekursi hingga seluruh data terurut. Pengurutan dilakukan dari nilai terkecil ke nilai terbesar. Proses berjalan secara otomatis sesuai algoritma yang diterapkan. Setelah selesai, array akan berada dalam kondisi terurut.

n. Hasil Program

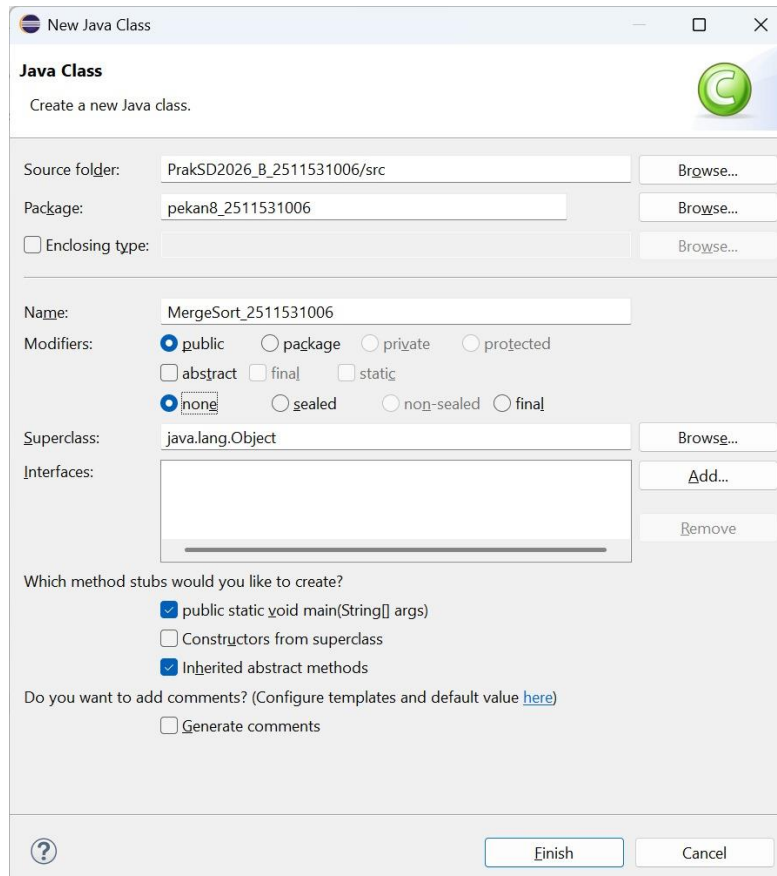


```
<terminated> QuickSort_2511531006 [Java Application] C:  
Data Sebelum diurutkan: 10 7 8 9 1 5  
Data Terurut Quick Sort : 1 5 7 8 9 10
```

Berdasarkan hasil pengujian yang dilakukan, program berhasil mengurutkan data menggunakan algoritma Quick Sort. Data yang awalnya belum terurut dapat disusun dari nilai terkecil hingga terbesar. Proses pengurutan dilakukan menggunakan teknik pivot dan rekursi. Program dapat berjalan dengan baik tanpa menghasilkan error. Hasil output yang diperoleh telah sesuai dengan tujuan praktikum yang diharapkan.

3. Program 3

a. Membuat Class Java



Pada tahap ini dibuat class dengan nama MergeSort_2511531006 di dalam package yang telah dibuat sebelumnya. Class merupakan tempat untuk menuliskan seluruh kode program yang akan dijalankan. Pembuatan class dilakukan melalui menu New → Class pada Eclipse. Saat membuat class dipilih opsi public static void main(String[] args) agar program dapat langsung dieksekusi. Setelah class berhasil dibuat, pengguna dapat mulai menuliskan kode program Merge Sort di dalamnya. Class ini nantinya menjadi pusat dari seluruh proses pengurutan data.

b. Menuliskan Package Program

```
1 package pekan8_2511531006;  
2
```

Pada bagian ini dituliskan package yang digunakan oleh program, yaitu pekan8_2511531006. Penulisan package harus sesuai dengan package yang telah dibuat sebelumnya agar tidak terjadi error saat program dijalankan. Package membantu mengelompokkan class berdasarkan fungsi atau praktikum yang sama. Dengan menggunakan package, file program menjadi lebih terstruktur dan mudah dikelola. Selain itu package juga membantu menghindari konflik nama class yang sama dalam project berbeda. Oleh karena itu penulisan package merupakan langkah penting dalam pembuatan program Java.

c. Mendeklarasikan Class

```
3 public class MergeSort_2511531006 {  
4
```

Pada bagian ini dibuat class utama bernama MergeSort_2511531006. Class digunakan sebagai wadah untuk menyimpan seluruh method, variabel, dan proses yang digunakan dalam program. Semua proses pengurutan data menggunakan Merge Sort akan ditulis di dalam class ini. Dalam pemrograman berorientasi objek, class merupakan komponen utama yang harus ada agar program dapat berjalan. Class juga berfungsi sebagai blueprint untuk pembuatan objek. Oleh karena itu seluruh kode program akan ditempatkan di dalam class ini.

d. Membuat Method Merge

```
4  
5 void merge_1006(int arr_1006[], int l_1006, int m_1006, int r_1006) {
```

Pada bagian ini dibuat method `merge_1006()` yang digunakan untuk menggabungkan dua subarray yang telah terurut menjadi satu array yang terurut. Method ini menerima parameter berupa array utama serta indeks kiri, tengah, dan kanan. Method merge merupakan inti dari algoritma Merge Sort karena berfungsi menyusun kembali data yang sebelumnya telah dipisahkan. Selama proses merge, elemen-elemen dari kedua subarray akan dibandingkan satu per satu. Nilai yang lebih kecil akan ditempatkan terlebih dahulu ke dalam array utama. Hasil akhirnya adalah bagian array yang sudah tersusun secara berurutan.

e. Menentukan Ukuran Subarray

```
int n1_1006 = m_1006 - l_1006 + 1;  
int n2_1006 = r_1006 - m_1006;
```

Pada bagian ini ditentukan jumlah elemen yang terdapat pada subarray kiri dan subarray kanan. Variabel `n1_1006` digunakan untuk menyimpan ukuran subarray kiri, sedangkan `n2_1006` digunakan untuk menyimpan ukuran subarray kanan. Informasi ukuran ini diperlukan untuk membuat array sementara yang akan digunakan saat proses merge. Dengan mengetahui jumlah elemen pada masing-masing subarray, program dapat melakukan proses penyalinan data dengan tepat. Perhitungan ukuran dilakukan berdasarkan posisi indeks kiri, tengah, dan kanan. Langkah ini menjadi dasar sebelum data dipindahkan ke array sementara.

f. Membuat Array Sementara

```
10 int L_1006[] = new int[n1_1006];  
11 int R_1006[] = new int[n2_1006];
```

Pada bagian ini dibuat dua array sementara yaitu L_1006 dan R_1006. Array sementara digunakan untuk menampung data dari subarray kiri dan subarray kanan. Penggunaan array sementara bertujuan agar data asli tidak langsung berubah saat proses penggabungan berlangsung. Dengan adanya array ini, proses perbandingan data menjadi lebih mudah dilakukan. Data yang tersimpan pada kedua array akan diproses dan dipindahkan kembali ke array utama secara berurutan. Langkah ini sangat penting dalam implementasi Merge Sort.

g. Menyalin Data ke Array Sementara

```
10      int L_1006[] = new int[n1_1006];  
11      int R_1006[] = new int[n2_1006];
```

Pada bagian ini data dari array utama disalin ke array sementara yang telah dibuat sebelumnya. Perulangan pertama digunakan untuk mengisi array kiri (L_1006), sedangkan perulangan kedua digunakan untuk mengisi array kanan (R_1006). Penyalinan data dilakukan agar proses merge dapat berjalan tanpa mengubah data asli secara langsung. Setiap elemen disalin sesuai posisi indeks masing-masing. Setelah semua data berhasil disalin, program siap melakukan proses perbandingan antar elemen. Tahap ini merupakan persiapan sebelum penggabungan data dilakukan.

h. Membandingkan Data Saat Merge

```
20      while (i_1006 < n1_1006 && j_1006 < n2_1006) {
```

Pada bagian ini dilakukan proses perbandingan antara elemen pada array kiri dan array kanan. Program akan mengambil elemen yang memiliki nilai lebih kecil untuk ditempatkan ke array utama. Jika nilai array kiri lebih kecil atau sama dengan array kanan, maka elemen kiri akan dipilih terlebih dahulu. Sebaliknya jika elemen kanan lebih kecil, maka elemen kanan yang akan dipindahkan. Proses ini berlangsung terus sampai salah satu array habis diproses. Dengan cara ini data dapat tersusun secara berurutan dari nilai terkecil ke terbesar.

i. Menyalin Sisa Data Array Kiri

```
31 while (i_1006 < n1_1006) {
```

Pada bagian ini program memeriksa apakah masih ada elemen yang tersisa pada array kiri setelah proses perbandingan selesai dilakukan. Jika masih terdapat data yang belum dipindahkan, maka data tersebut akan disalin ke array utama. Penyalinan dilakukan menggunakan perulangan while sampai seluruh elemen selesai diproses. Langkah ini penting agar tidak ada data yang hilang selama proses merge berlangsung. Semua elemen yang tersisa harus tetap dimasukkan ke hasil akhir. Dengan demikian hasil penggabungan menjadi lengkap dan benar.

j. Menyalin Sisa Data Array Kanan

```
31 while (i_1006 < n1_1006) {
```

Pada bagian ini program memeriksa apakah masih terdapat elemen yang tersisa pada array kanan. Jika ada, maka elemen tersebut akan dipindahkan ke array utama satu per satu. Proses dilakukan sampai seluruh elemen array kanan selesai disalin. Langkah ini diperlukan untuk memastikan seluruh data ikut masuk ke hasil akhir penggabungan. Tanpa proses ini sebagian data dapat terlewat sehingga hasil sorting menjadi tidak lengkap. Oleh karena itu bagian ini merupakan bagian penting dari proses merge.

k. Membuat Method Sort

```
43 void sort_1006(int arr_1006[], int l_1006, int r_1006) {
```

Pada bagian ini dibuat method `sort_1006()` yang berfungsi menjalankan algoritma Merge Sort secara keseluruhan. Method ini bekerja menggunakan konsep rekursif, yaitu memanggil dirinya sendiri sampai ukuran array menjadi sangat kecil. Array akan dibagi menjadi dua bagian secara terus-menerus hingga setiap bagian hanya memiliki satu elemen. Setelah proses pembagian selesai, method merge akan digunakan untuk menggabungkan kembali data yang telah dipisahkan. Proses ini dilakukan berulang sampai seluruh array menjadi teratur. Method sort merupakan pengendali utama dalam algoritma Merge Sort.

1. Menentukan Titik Tengah Array

```
46 int m_1006 = l_1006 + (r_1006 - l_1006) / 2;
```

Pada bagian ini program menentukan posisi tengah array yang digunakan untuk membagi data menjadi dua bagian. Nilai tengah dihitung menggunakan indeks kiri dan indeks kanan array. Titik tengah ini menjadi batas antara subarray kiri dan subarray kanan. Pembagian array menjadi dua bagian merupakan konsep utama dari algoritma Merge Sort. Setelah titik tengah diperoleh, kedua bagian akan diproses kembali secara rekursif. Dengan cara ini proses pengurutan dapat dilakukan secara lebih efisien.

1. Membuat Method Print Array

```
56 static void printArray_1006(int arr_1006[]) {
```

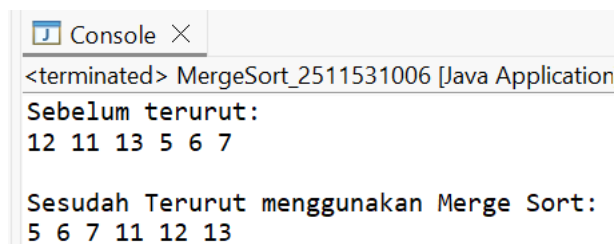
Pada bagian ini dibuat method `printArray_1006()` yang digunakan untuk menampilkan isi array ke layar. Method ini menerima parameter berupa array yang akan dicetak. Seluruh elemen array dibaca menggunakan perulangan `for` kemudian ditampilkan menggunakan perintah `System.out.print()`. Method ini digunakan sebelum dan sesudah proses sorting dilakukan. Tujuannya adalah untuk memperlihatkan perubahan urutan data setelah algoritma Merge Sort dijalankan. Dengan demikian hasil pengurutan dapat diamati dengan jelas.

m. Membuat Data dan Menjalankan Merge Sort

```
63 public static void main(String[] args_1006) {  
    int arr_1006[] = {12, 11, 13, 5, 6, 7};  
  
66 MergeSort_2511531006 ob_1006 = new MergeSort_2511531006();  
67 ob_1006.sort_1006(arr_1006, 0, arr_1006.length - 1);  
}
```

Pada bagian ini dibuat array yang berisi data yang akan diurutkan menggunakan algoritma Merge Sort. Data yang digunakan masih dalam kondisi belum terurut sehingga dapat digunakan untuk menguji program. Selanjutnya dibuat objek dari class `MergeSort_2511531006` untuk memanggil method `sort_1006()`. Method tersebut kemudian dijalankan untuk melakukan proses pengurutan data. Program akan membagi array menjadi beberapa bagian kecil kemudian menggabungkannya kembali dalam keadaan terurut. Setelah proses selesai, hasil pengurutan akan ditampilkan ke layar.

n. Hasil Program

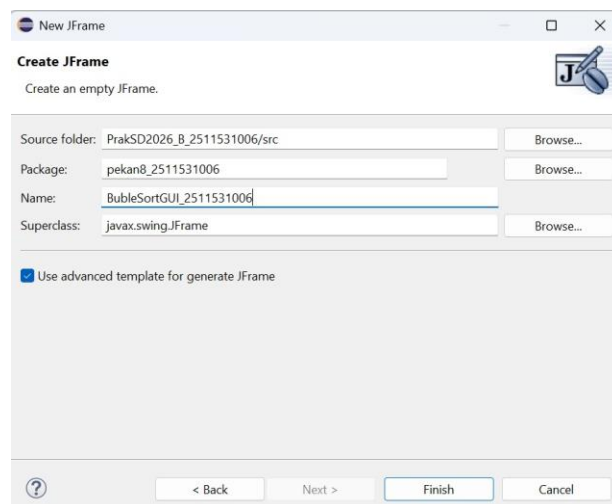


```
<terminated> MergeSort_2511531006 [Java Application]  
Sebelum terurut:  
12 11 13 5 6 7  
  
Sesudah Terurut menggunakan Merge Sort:  
5 6 7 11 12 13
```

Berdasarkan hasil pengujian yang dilakukan, program berhasil mengurutkan data menggunakan algoritma Merge Sort dengan baik. Data yang awalnya berada dalam kondisi acak dapat diurutkan dari nilai terkecil hingga nilai terbesar. Proses pengurutan dilakukan melalui dua tahap utama, yaitu membagi array menjadi beberapa bagian kecil dan menggabungkannya kembali dalam keadaan terurut. Program dapat dijalankan tanpa menghasilkan error dan memberikan hasil yang sesuai dengan teori Merge Sort. Dari hasil tersebut dapat disimpulkan bahwa implementasi algoritma Merge Sort pada program telah berjalan dengan benar dan sesuai tujuan praktikum.

4. Program 4

a. Membuat JFrame



Pada tahap ini dibuat sebuah JFrame dengan nama BubleSortGUI_1006. JFrame digunakan sebagai jendela utama yang akan menampilkan seluruh komponen GUI program. Pembuatan JFrame dilakukan melalui menu New → JFrame pada Eclipse. JFrame berfungsi sebagai wadah untuk menempatkan tombol, label, text field, dan komponen lainnya. Setelah JFrame selesai dibuat, program siap digunakan untuk membuat tampilan Bubble Sort secara visual.

b. Mengimpor Library yang Digunakan

```

3 import java.awt.*;
4
5 import javax.swing.*;

```

Pada bagian ini dilakukan import library yang dibutuhkan untuk membuat tampilan GUI. Library java.awt digunakan untuk mengatur layout, warna, font, dan ukuran komponen. Sedangkan javax.swing digunakan untuk membuat komponen seperti JFrame, JButton, JLabel, JTextArea, dan lain-lain. Tanpa library tersebut program GUI tidak dapat dijalankan. Semua komponen visual yang digunakan berasal dari library ini.

c. Mendeklarasikan Variabel Program

```

9 private static final long serialVersionUID = 1L;
10 private int[] array_1006;
11 private JLabel[] labelArray_1006;
12 private JButton stepButton_1006;
13 private JButton resetButton_1006;
14 private JButton setButton_1006;
15 private JTextField inputField_1006;
16 private JPanel panelArray_1006;
17 private JTextArea stepArea_1006;
18 private int i_1006 = 0;
19 private int j_1006 = 0;
20 private boolean sorting_1006 = false;
21 private int stepCount_1006 = 1;

```

Pada bagian ini dibuat berbagai variabel yang digunakan selama program berjalan. Variabel tersebut digunakan untuk menyimpan data array, tombol, label, text field, panel, dan area log. Selain itu terdapat variabel untuk mengontrol proses Bubble Sort seperti nilai i, j, dan jumlah langkah. Variabel ini akan digunakan oleh method-

d. Membuat Konstruktor JFrame

Pada bagian ini dibuat konstruktor BubbleSortGUI_1006() yang akan dijalankan saat objek dibuat. Konstruktor digunakan untuk mengatur ukuran jendela, judul program, posisi jendela, serta layout yang digunakan. Selain itu seluruh komponen GUI juga dibuat di dalam konstruktor. Dengan adanya konstruktor, tampilan program akan langsung muncul saat dijalankan. Konstruktor menjadi bagian utama dalam pembuatan GUI.

```

23 public BubbleSortGUI_1006() {

```

e. Mengatur Properti JFrame

```

25 setTitle("Bubble Sort Langkah per Langkah");
26 setSize(750, 400);
27 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
28 setLocationRelativeTo(null);
29 setLayout(new BorderLayout());

```

Pada bagian ini dilakukan pengaturan tampilan utama JFrame. Program memberikan judul jendela, menentukan ukuran jendela, menentukan operasi saat tombol close ditekan, serta mengatur posisi jendela agar muncul di tengah layar. Layout yang digunakan adalah BorderLayout. Pengaturan ini membuat tampilan program menjadi lebih rapi dan mudah digunakan.

f. Membuat Panel Input

```

30
31 JPanel inputPanel_1006 = new JPanel(new FlowLayout());
32
33 inputField_1006 = new JTextField(30);
34 setButton_1006 = new JButton("Set Array");

```

Pada bagian ini dibuat panel input yang digunakan untuk menerima data angka dari pengguna. Panel menggunakan FlowLayout agar komponen tersusun secara horizontal. Pengguna dapat memasukkan angka yang dipisahkan dengan tanda koma. Tombol Set Array digunakan untuk mengubah input menjadi array yang akan diproses oleh Bubble Sort. Panel ini menjadi tempat interaksi awal pengguna dengan program.

g. Membuat Panel Tampilan Array

```

40
41 panelArray_1006 = new JPanel();
42 panelArray_1006.setLayout(new FlowLayout());

```

Pada bagian ini dibuat panel yang digunakan untuk menampilkan elemen-elemen array. Setiap elemen array nantinya akan ditampilkan dalam bentuk label. Panel ini akan diperbarui setiap kali terjadi perubahan data selama proses sorting. Dengan adanya panel ini pengguna dapat melihat proses Bubble Sort secara visual. Tampilan data menjadi lebih mudah dipahami.

h. Membuat Tombol Kontrol

```
46     stepButton_1006 = new JButton("Langkah Selanjutnya");
47     resetButton_1006 = new JButton("Reset");
48
49     stepButton_1006.setEnabled(false);
50
```

Pada bagian ini dibuat tombol untuk mengontrol jalannya program. Tombol Langkah Selanjutnya digunakan untuk menjalankan proses Bubble Sort satu langkah demi satu langkah. Tombol Reset digunakan untuk menghapus seluruh data dan mengembalikan kondisi program ke awal. Tombol langkah awalnya dinonaktifkan hingga data dimasukkan. Dengan adanya tombol ini pengguna dapat mengontrol proses sorting.

i. Membuat Area Log Langkah Sorting

```
55     stepArea_1006.setEditable(false);
56     stepArea_1006.setFont(
57         new Font("Monospaced", Font.PLAIN, 14));
```

Pada bagian ini dibuat JTextArea yang digunakan untuk menampilkan setiap langkah Bubble Sort. Area ini akan menampilkan proses pertukaran data yang terjadi. Font Monospaced digunakan agar tampilan lebih rapi dan mudah dibaca. Dengan adanya area log, pengguna dapat melihat proses pengurutan secara detail. Informasi langkah akan bertambah setiap kali tombol langkah ditekan.

j. Menambahkan Komponen ke JFrame

```
62         add(inputPanel_1006, BorderLayout.NORTH);
63         add(panelArray_1006, BorderLayout.CENTER);
64         add(controlPanel_1006, BorderLayout.SOUTH);
65         add(scrollPane_1006, BorderLayout.EAST);
66     }
```

Pada bagian ini seluruh komponen yang telah dibuat ditambahkan ke JFrame sesuai posisi BorderLayout. Panel input ditempatkan di bagian atas, panel array di tengah, tombol kontrol di bawah, dan area log di samping kanan. Pengaturan ini membuat tampilan lebih terstruktur dan mudah digunakan. Semua komponen akan tampil dalam satu jendela utama.

k. Menambahkan Event Listener

```
67         setButton_1006.addActionListener(
68             e -> setArrayFromInput());
69
70         stepButton_1006.addActionListener(
71             e -> performStep());
72
73         resetButton_1006.addActionListener(
74             e -> reset());
75     }
```

Pada bagian ini ditambahkan event listener pada setiap tombol. Event listener berfungsi untuk menjalankan method tertentu ketika tombol ditekan pengguna. Tombol Set Array akan memanggil method setArrayFromInput(), tombol Langkah Selanjutnya memanggil performStep(), dan tombol Reset memanggil reset(). Dengan adanya event listener program dapat merespon aksi pengguna.

l. Membuat Method setArrayFromInput()

```
76     -
77     private void setArrayFromInput() {
```

Pada bagian ini dibuat method yang digunakan untuk mengambil input dari pengguna. Data yang dimasukkan dipisahkan menggunakan tanda koma kemudian diubah menjadi array integer. Method juga melakukan pengecekan agar input yang dimasukkan benar-benar

berupa angka. Setelah data berhasil dibaca, elemen array akan ditampilkan ke panel. Method ini menjadi awal proses Bubble Sort.

m. Membuat Label Array

```
102 labelArray_1006[k_1006] =new JLabel(String.valueOf(array_1006[k_1006]));
```

Pada bagian ini dibuat label untuk setiap elemen array yang dimasukkan pengguna. Label digunakan untuk menampilkan nilai array secara visual pada layar. Setiap label diberi ukuran, warna latar, dan border agar lebih mudah dilihat. Label akan berubah warna saat proses sorting berlangsung. Dengan demikian proses Bubble Sort dapat diamati secara langsung.

n. Membuat Method performStep()

```
115  
116 private void performStep() {
```

Pada bagian ini dibuat method yang digunakan untuk menjalankan satu langkah Bubble Sort. Method akan membandingkan dua elemen yang berdekatan. Jika elemen kiri lebih besar maka dilakukan pertukaran posisi. Setiap proses akan dicatat ke area log dan ditampilkan pada layar. Method ini memungkinkan pengguna melihat proses sorting secara bertahap.

o. Melakukan Pertukaran Data (Swap)

```
130 int temp_1006 =array_1006[j_1006];  
131 array_1006[j_1006] =array_1006[j_1006 + 1];  
132 array_1006[j_1006 + 1] =temp_1006;
```

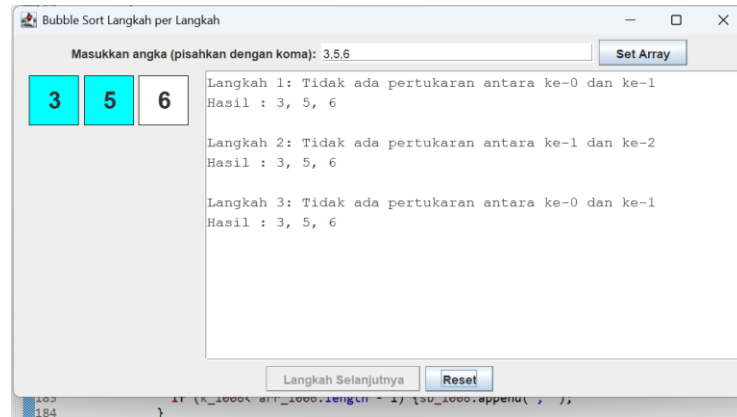
Pada bagian ini dilakukan pertukaran posisi data jika elemen kiri lebih besar dari elemen kanan. Pertukaran dilakukan menggunakan variabel sementara. Setelah pertukaran selesai, warna label berubah menjadi merah sebagai tanda adanya swap. Langkah ini merupakan inti dari algoritma Bubble Sort. Data yang lebih besar akan bergerak ke bagian kanan array.

p. Membuat Method Pendukung dan Hasil Program

```
154 private void updateLabels() {
155     for (int k_1006 = 0; k_1006 < array_1006.length; k_1006++) {
156         labelArray_1006[k_1006].setText(String.valueOf(array_1006[k_1006]));
157     }
158 }
159
160 private void resetHighlights() {
161     for (JLabel label_1006 : labelArray_1006) {
162         label_1006.setBackground(Color.WHITE);
163     }
164 }
165
166 private void reset() {
167     inputField_1006.setText("");
168     panelArray_1006.removeAll();
169     panelArray_1006.revalidate();
170     panelArray_1006.repaint();
171     stepArea_1006.setText("");
172     stepButton_1006.setEnabled(false);
173     sorting_1006 = false;
174     i_1006 = 0;
175     j_1006 = 0;
176     stepCount_1006 = 1;
177 }
178
179 private String arrayToString(int[] arr_1006) {
180     StringBuilder sb_1006 = new StringBuilder();
181     for (int k_1006 = 0; k_1006 < arr_1006.length; k_1006++) {
182         sb_1006.append(arr_1006[k_1006]);
183         if (k_1006 < arr_1006.length - 1) {sb_1006.append(", ");
184         }
185     }
186     return sb_1006.toString();
187 }
188
189 }
```

Pada bagian ini dibuat method `updateLabels()`, `resetHighlights()`, `reset()`, dan `arrayToString()`. Method tersebut digunakan untuk memperbarui tampilan array, mengembalikan warna label, mereset program, serta mengubah array menjadi teks. Setelah seluruh proses Bubble Sort selesai, data akan tampil dalam keadaan terurut. Program berhasil menunjukkan proses Bubble Sort secara visual dan langkah demi langkah. Hasil ini memudahkan pengguna memahami cara kerja algoritma Bubble Sort.

r. Hasil Output Program



Berdasarkan hasil pengujian yang dilakukan, program GUI Bubble Sort berhasil berjalan dengan baik. Pengguna dapat memasukkan data sendiri, melihat proses perbandingan dan pertukaran data secara bertahap, serta melihat hasil akhir pengurutan. Tampilan GUI membuat proses Bubble Sort lebih mudah dipahami dibandingkan hanya menggunakan output console. Program juga dapat menampilkan setiap langkah sorting secara rinci sehingga cocok digunakan sebagai media pembelajaran algoritma Bubble Sort.

2.2 Langkah Langkah Tugas Praktikum

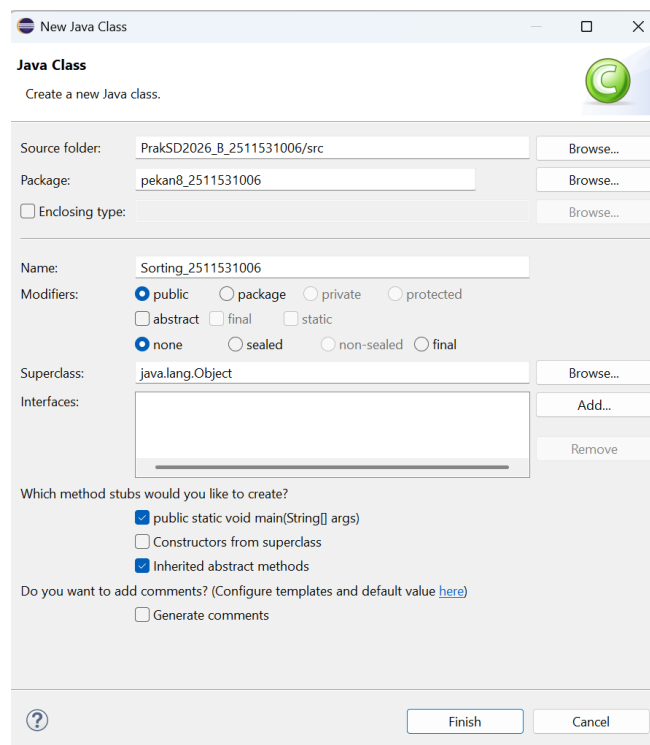
Saya memilih algoritma Quick Sort karena beberapa alasan berikut:

1. Quick Sort itu mudah dipahami dan implementasinya tidak terlalu rumit. Struktur kodenya menggunakan konsep rekursif yang sudah dipelajari sebelumnya dalam mata kuliah Struktur Data.
2. karena soal meminta pengurutan berdasarkan durasi (angka), Quick Sort sangat cocok digunakan. Perbandingan angka bulat ($<$ dan $>$) lebih mudah dibanding mengurutkan berdasarkan huruf (String).
3. Quick Sort sangat efisien untuk mengurutkan data dalam jumlah sedikit hingga sedang, seperti 7-20 lagu sesuai ketentuan soal. Kompleksitas waktunya rata-rata $O(n \log n)$ dianggap sudah baik untuk kasus ini.

4. algoritma ini bekerja secara in-place pada array, artinya tidak membutuhkan memori tambahan yang besar. Sesuai dengan penggunaan array dataLagu_1006 berkapasitas 20 lagu.

1. Program 1

a. Membuat Class Java



Pada tahap ini dibuat class dengan nama `Sorting_2511531006` pada package `pekan8_2511531006`. Class ini digunakan sebagai tempat menuliskan seluruh kode program sorting playlist lagu. Pembuatan class dilakukan melalui menu `New → Class` pada Eclipse. Saat membuat class dipilih method `public static void main(String[] args)` agar program dapat dijalankan. Setelah proses selesai, class siap digunakan untuk implementasi algoritma sorting.

b. Membuat Class Lagu

Pada bagian ini dibuat class Lagu yang berfungsi untuk menyimpan data setiap lagu. Class ini memiliki atribut judul lagu, penyanyi, dan durasi lagu. Data tersebut nantinya akan disimpan ke dalam array playlist. Class Lagu digunakan agar data lebih terstruktur dan mudah dikelola. Setiap objek Lagu akan mewakili satu data lagu pada playlist.

```
5  static class Lagu {  
6      String judul;  
7      String penyanyi;  
8      int durasi;
```

c. Membuat Konstruktor Lagu

```
10 Lagu(String judul, String penyanyi, int durasi) {  
11     this.judul = judul;  
12     this.penyanyi = penyanyi;  
13     this.durasi = durasi;  
14 }  
15 }
```

Pada bagian ini dibuat konstruktor untuk mengisi data lagu saat objek dibuat. Konstruktor menerima parameter berupa judul, penyanyi, dan durasi lagu. Nilai parameter kemudian disimpan ke atribut class menggunakan keyword this. Dengan adanya konstruktor, proses pembuatan objek lagu menjadi lebih mudah dan singkat. Setiap lagu dapat langsung memiliki data lengkap saat dibuat.

d. Membuat Array Data Lagu

```
16  
17 Lagu[] dataLagu_1006 = new Lagu[7];  
18 int n = 0;
```

Pada bagian ini dibuat array untuk menyimpan data lagu. Array memiliki kapasitas maksimal 20 lagu sesuai ketentuan tugas praktikum. Variabel n digunakan untuk menyimpan jumlah data lagu yang sedang digunakan. Data lagu nantinya akan dimasukkan

ke dalam array ini sebelum dilakukan proses sorting. Array menjadi media utama penyimpanan data playlist.

e. Membuat Method Input Data

```
21⊖ void inputData_1006() {
```

Pada bagian ini dibuat method `inputData_1006()` yang digunakan untuk mengisi data awal playlist lagu. Data yang dimasukkan terdiri dari judul lagu, penyanyi, dan durasi lagu. Jumlah lagu yang dimasukkan sebanyak tujuh data sesuai syarat minimal pada soal. Setelah seluruh data dimasukkan, variabel `n` diisi dengan jumlah data yang tersedia. Method ini digunakan sebelum proses sorting dijalankan.

f. Mengisi Data Lagu ke Dalam Array

```
22 dataLagu_1006[0] = new Lagu("Mio Cristo Piange Diamanti", "Il Giardino Dei Semplici", 270);  
23 dataLagu_1006[1] = new Lagu("La Rumba Del Perdon", "Diego Torres", 252);  
24 dataLagu_1006[2] = new Lagu("La Perla", "Chayanne", 196);  
25 dataLagu_1006[3] = new Lagu("Bambino", "Sergio Cammare", 150);  
26 dataLagu_1006[4] = new Lagu("Malaguena Salerosa", "Luis Miguel", 210);  
27 dataLagu_1006[5] = new Lagu("Corazon Espinado", "Santana", 245);  
28 dataLagu_1006[6] = new Lagu("Amarte Duele", "Jesse McCartney", 205);  
29  
30 n = 7;
```

Pada bagian ini dilakukan pengisian data lagu ke dalam array `dataLagu_1006`. Setiap indeks array berisi objek `Lagu` yang memiliki judul, penyanyi, dan durasi. Data ini nantinya akan digunakan sebagai bahan pengujian algoritma sorting. Pengisian dilakukan secara manual menggunakan keyword `new Lagu()`. Setelah selesai, seluruh data siap diproses.

g. Membuat Method Tampil Data

```
32  
33⊖ void tampilData 1006(String kondisi) {
```

Pada bagian ini dibuat method `tampilData_1006()` yang digunakan untuk menampilkan data playlist ke layar. Method menerima parameter berupa keterangan kondisi data. Seluruh data lagu akan ditampilkan menggunakan perulangan `for`. Informasi yang

ditampilkan meliputi nomor urut, judul lagu, dan durasi lagu. Method ini digunakan sebelum dan sesudah proses sorting.

h. Membuat Method Shell Sort

```
39  
40 void shellSort_1006() {
```

Pada bagian ini dibuat method shellSort_1006() yang digunakan untuk mengurutkan data berdasarkan judul lagu dari A sampai Z. Algoritma Shell Sort bekerja dengan menggunakan jarak atau gap yang terus diperkecil. Data dibandingkan dan dipindahkan sesuai urutan alfabet. Proses dilakukan berulang sampai gap bernilai satu. Setelah selesai, seluruh data akan tersusun berdasarkan judul lagu.

i. Membuat Perulangan Shell Sort

```
41 for (int gap = n / 2; gap > 0; gap /= 2) {
```

Pada bagian ini dibuat perulangan untuk mengatur nilai gap pada algoritma Shell Sort. Nilai gap awal diperoleh dari jumlah data dibagi dua. Setelah satu tahap selesai, gap akan diperkecil hingga bernilai satu. Semakin kecil gap maka proses pengurutan semakin detail. Perulangan ini menjadi dasar kerja algoritma Shell Sort.

j. Membuat Method Quick Sort

```
56  
57 void quickSort_1006(int low, int high) {
```

Pada bagian ini dibuat method quickSort_1006() yang digunakan untuk mengurutkan data berdasarkan durasi lagu dari yang tercepat ke yang terlama. Quick Sort bekerja dengan memilih sebuah pivot lalu membagi data menjadi dua bagian. Data yang lebih kecil ditempatkan di kiri pivot dan data yang lebih besar di kanan pivot. Proses dilakukan secara rekursif hingga seluruh data terurut. Algoritma ini dikenal memiliki performa yang cepat.

k. Membuat Method Partition

```
64  
65 int partition_1006(int low, int high) {
```

Pada bagian ini dibuat method `partition_1006()` yang digunakan untuk menentukan posisi pivot pada Quick Sort. Pivot dipilih dari elemen terakhir array. Data yang lebih kecil dari pivot dipindahkan ke bagian kiri. Sedangkan data yang lebih besar tetap berada di bagian kanan. Method ini menentukan titik pembagian sebelum proses rekursif dilakukan.

l. Membuat Method Merge Sort

```
81  
82 void mergeSort_1006(int left, int right) {
```

Pada bagian ini dibuat method `mergeSort_1006()` yang digunakan untuk mengurutkan data berdasarkan judul lagu secara alfabetis. Algoritma Merge Sort bekerja dengan membagi array menjadi beberapa bagian kecil. Setelah itu setiap bagian akan digabungkan kembali dalam kondisi terurut. Proses dilakukan secara rekursif hingga seluruh data tersusun dengan benar. Algoritma ini memiliki kompleksitas yang stabil.

m. Membuat Method Merge

```
90  
91 void merge_1006(int left, int mid, int right) {  
92
```

Pada bagian ini dibuat method `merge_1006()` yang digunakan untuk menggabungkan dua bagian array yang sudah terurut. Data dari array kiri dan kanan dibandingkan satu per satu. Elemen yang lebih kecil akan dimasukkan terlebih dahulu ke array utama. Proses dilakukan hingga seluruh elemen berhasil digabungkan. Method ini merupakan inti dari algoritma Merge Sort.

n. Membuat Method Main

```
14  
15 public static void main(String[] args) {
```

Pada bagian ini dibuat method utama yang digunakan untuk menjalankan program. Method main membuat objek dari class Sorting_2511531006 dan Scanner untuk menerima input pengguna. Pengguna dapat memilih algoritma sorting yang ingin digunakan. Setelah pilihan dimasukkan, program akan menjalankan algoritma yang sesuai. Method main menjadi titik awal eksekusi program.

o. Memilih Algoritma Sorting

```
126 if (pilihan == 1) {  
127     program.shellSort_1006();  
128     program.tampilData_1006("Data Setelah Shell Sort (Judul A-Z)");  
129 } else if (pilihan == 2) {  
130     program.quickSort_1006(0, program.n - 1);  
131     program.tampilData_1006("Data Setelah Quick Sort (Durasi Asc)");  
132 } else if (pilihan == 3) {  
133     program.mergeSort_1006(0, program.n - 1);  
134     program.tampilData_1006("Data Después Merge Sort (Judul A-Z)");  
135 }  
136 , }
```

Pada bagian ini pengguna diberikan pilihan algoritma yang akan digunakan. Jika memilih angka 1 maka program menjalankan Shell Sort. Jika memilih angka 2 maka program menjalankan Quick Sort. Sedangkan angka 3 digunakan untuk menjalankan Merge Sort. Setelah sorting selesai, data hasil pengurutan akan ditampilkan ke layar.

p. Hasil Program

Berdasarkan hasil pengujian yang dilakukan, program berhasil mengurutkan data playlist lagu menggunakan algoritma sorting yang dipilih pengguna. Data lagu dapat ditampilkan sebelum dan sesudah proses sorting sehingga perubahan urutan dapat terlihat dengan jelas. Program telah memenuhi ketentuan tugas karena menggunakan array, memiliki minimal tujuh data lagu, dan menerapkan aturan penamaan sesuai NIM. Selain itu program juga

menyediakan tiga pilihan algoritma yaitu Shell Sort, Quick Sort, dan Merge Sort. Hasil yang diperoleh telah sesuai dengan tujuan praktikum yang diharapkan.

```
Console X
<terminated> Sorting_2511531006 [Java Application] C:\Users
=== Sorting Playlist NIM: 2511531006 ===

Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
|
Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik
4. Bambino - 150 detik
5. Malaguena Salerosa - 210 detik
6. Corazon Espinado - 245 detik
7. Amarte Duele - 205 detik

Data Setelah Quick Sort (Durasi Asc):
1. Bambino - 150 detik
2. La Perla - 196 detik
3. Amarte Duele - 205 detik
4. Malaguena Salerosa - 210 detik
5. Corazon Espinado - 245 detik
6. La Rumba Del Perdon - 252 detik
7. Mio Cristo Piange Diamanti - 270 detik
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, program sorting playlist lagu berhasil dibuat menggunakan bahasa pemrograman Java. Program mampu mengelola data lagu yang terdiri dari judul, penyanyi, dan durasi lagu menggunakan struktur array. Selain itu, program juga berhasil menerapkan tiga algoritma pengurutan yaitu Shell Sort, Quick Sort, dan Merge Sort sesuai dengan ketentuan praktikum. Pengguna dapat memilih algoritma yang ingin digunakan dan melihat hasil pengurutan data sebelum maupun sesudah proses sorting dilakukan. Secara keseluruhan, program dapat berjalan dengan baik dan telah memenuhi tujuan praktikum yang diharapkan.

3.2 Saran

Dalam pengembangan selanjutnya, program dapat ditambahkan fitur input data lagu secara dinamis sehingga pengguna dapat menambahkan, mengubah, maupun menghapus data lagu sesuai kebutuhan. Selain itu, tampilan program juga dapat dikembangkan menggunakan Graphical User Interface (GUI) agar lebih menarik dan mudah digunakan. Program juga dapat dikembangkan dengan menambahkan metode pencarian data serta penyimpanan data ke file atau database. Dengan adanya pengembangan tersebut, program akan menjadi lebih interaktif dan memiliki fungsi yang lebih lengkap.

DAFTAR PUSTAKA

1. Data Structures and Algorithms in Java, Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th Edition). Wiley.
2. Introduction to Java Programming and Data Structures, Liang, Y. D. (2019). *Introduction to Java Programming and Data Structures* (12th Edition). Pearson.
3. Oracle. *Java Documentation*.
[Oracle Java Documentation](#)
4. Algorithms, Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th Edition). Addison-Wesley.
5. Data Structures and Algorithm Analysis in Java, Weiss, M. A. (2012). *Data Structures and Algorithm Analysis in Java* (3rd Edition). Pearson.

1.2

1.3

